

Git is a free and open source version control system . What is a version control?

Version Control is the management of changes to (documents , computer programs , large websites) so this means that if uploaded our code to git and changed it we can see every change that er did ad what time we did it so kind of keeps tracks of our changes .

Terms of Git:

- 1- Directory ⇒ Which is like a folder on the computer**
- 2- Terminal or Command Line ⇒ interface for texting commands**
- 3- CLI ⇒ Command line interface**
- 4- Cd⇒ Change Directory**
- 5- Code Editor ⇒ Word processor for writing our code**
- 6- Repository⇒ project or the folder/place where our project is kept.**
- 7- Github⇒ A website to host your repositories online to code with other people and sort of like a portfolio for potential employers.**

Git Commands

clone⇒ Bring a repository that is hosted somewhere like github into local machine

add⇒ track files and changes in Git

commit ⇒ save files in git

push ⇒ upload Git commits into a remote repo like Github

pull⇒ download changes from a remote repo to local machine so opposite of push

SSH Keys: (Secure Shell) keys ⇒ is what connects Git and Github to prove that i am the owner of the repo .

How to use ?

ssh-keygen command then -t rsa -b 4096 -c "email@example.com"

this is how to get a new key ⇒ ssh-keygen -t rsa -b 4096 -C "your_email@example.com"

1. **t rsa**: Specifies the type of key to create (in this case, RSA)
 - RSA is a widely used public-key cryptosystem
 - Other options include **ed25519** (more modern and secure)
2. **b 4096**: Specifies the number of bits in the key
 - 4096 bits is currently considered very secure
 - The default is 2048 bits (still secure but 4096 is better)
3. **c "your_email@example.com"**: Adds a comment to help identify the key
 - This is typically your email address
 - It's added as a label at the end of the public key
 - Doesn't affect security, just helps you identify keys

Step-by-Step Usage

1. Generate SSH Key Pair

Run the command in your terminal:

```
bash
```

Copy

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

You'll be prompted for:

- Where to save the key (default is `~/.ssh/id_rsa`)
- A passphrase (optional but recommended for extra security)

This creates two files:

- **id_rsa**: Your private key (NEVER share this)
- **id_rsa.pub**: Your public key (this is what you share with GitHub)

2. Add SSH Key to SSH Agent

Start the SSH agent and add your key:

bash

Copy

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_rsa
```

3. Add Public Key to GitHub

1. Copy your public key to clipboard:

bash

Copy

```
cat ~/.ssh/id_rsa.pub | pbcopy # Mac  
# or  
cat ~/.ssh/id_rsa.pub | xclip -sel clip # Linux  
# or just open the file and copy manually
```

2. Go to GitHub → Settings → SSH and GPG keys → New SSH key
3. Paste your public key
4. Give it a descriptive title (e.g., "My Work Laptop")

4. Test the Connection

bash

Copy

```
ssh -T git@github.com
```

You should see a message like:

Copy

```
Hi username! You've successfully authenticated, but GitHub does not provide  
shell access.
```

Why This Works

- When you push to GitHub, your local Git uses your private key to create a digital signature
- GitHub verifies this signature using your public key
- Since only you have the private key, GitHub can be sure it's really you

- This is more secure than password authentication and more convenient

Best Practices

1. Use a strong passphrase for your private key
2. Don't share your private key with anyone
3. Use different keys for different machines/services
4. Consider using **ed25519** instead of RSA for newer systems:

bash

Copy

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

2. Faster Ways to Find SSH Keys

Option 1: Check Default Locations

Run these **quick** commands in CMD:

cmd

Copy

```
dir "%USERPROFILE%\.ssh" /a
dir "C:\\Program Files\\Git\\usr\\.ssh" /a
```

- **/a** shows hidden files (like **.ssh**).
- Most keys are in:
 - **C:\\Users\\[YourUsername]\\\.ssh** (default)
 - **C:\\Program Files\\Git\\usr\\.ssh** (if using Git Bash)

Option 2: Search Only Your User Folder

cmd

Copy

```
where /R "%USERPROFILE%" id_rsa*
```

- Searches only your profile (faster than scanning **c:**).

3. If Keys Don't Exist

Generate new ones (safe, even if old keys exist):

cmd

Copy

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

- Press **Enter 3 times** to save to `%USERPROFILE%\.ssh\id_rsa`.
-

4. Verify Keys

cmd

Copy

```
type "%USERPROFILE%\.ssh\id_rsa.pub"
```

- If you see a key starting with `ssh-rsa AAAAB3...`, it worked!
-

What Happened?

- **where** `/R C:\` scans **every file on C:**, which is slow.
 - It's harmless but inefficient—like searching your entire house for keys instead of checking the key rack first.
-

Next Steps

1. Add the key to GitHub:

cmd

Copy

```
type "%USERPROFILE%\.ssh\id_rsa.pub" | clip
```

Then paste at github.com/settings/ssh/new.

2. Test:

cmd

Copy

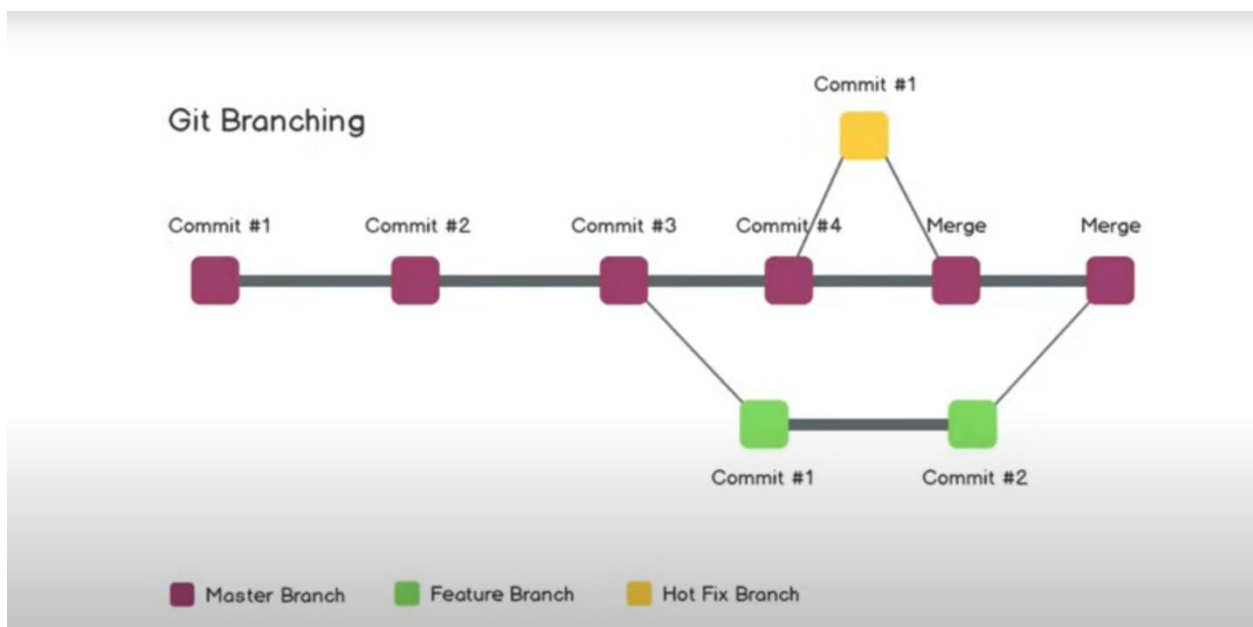
```
ssh -T git@github.com
```

Git branching

Master branch ⇒ main or default branch

if we have other branch and committed any changes we won't be able to see changes if we switched to master because it's only visible at the same branch. each individual branch knows changes on it

So branches is useful when we are working as multiple people in the same project



git branch ⇒ makes a new branch

git checkout -b (name)

if we did git branch gives the branches we are working on the highlighted on is what we are in and if we want to exit we type q

checkout branch switches between branches.

git status⇒ tells me what i did and the status of my repo files

git diff⇒ what changes have been made by comparing two codes

git merge ⇒ combines changes from one branch into another, integrating different lines of development into a single branch. It's used to bring feature branches, bug fixes, or updates into your main codebase

PR ⇒ pull request when we pull some code from a branch to another one by using github from compare an pull request which enable all people working together to make a comment and for owner to reply .

we can put description with list if changes we did .

we can do a comment for a single line of code which appears on it by + button

I can merge from the merge button.

git merge --abort Cancels a merge (if conflicts occur).

git merge --no-ff Forces a merge commit (even if fast-forward is possible).

git branch -d name ⇒ deletes the branch if we merged everything to the master

for modified files we can use **git commit -am** “added smth”

Merge conflict: we have to commit first or stash (which means saving temporarily and then merge).

we can fix it in the code , CMD or github , but easiest inside the code .

Undoing Git

git reset alone or with the name of file to remove

git reset HEAD ⇒ the last commit

git reset HEAD~1 ⇒ the one before

git log shows all the commits

git reset —hard (put commit hash)

Forking

we don't have access to do a PR pr branching so we fork other people repo to do whatever i want because forking gets the exact same repo.

we can the do a pill request and upload it .